

CS675

Computer Graphics

Modeling-Viewing Pipeline

Parag Chaudhuri

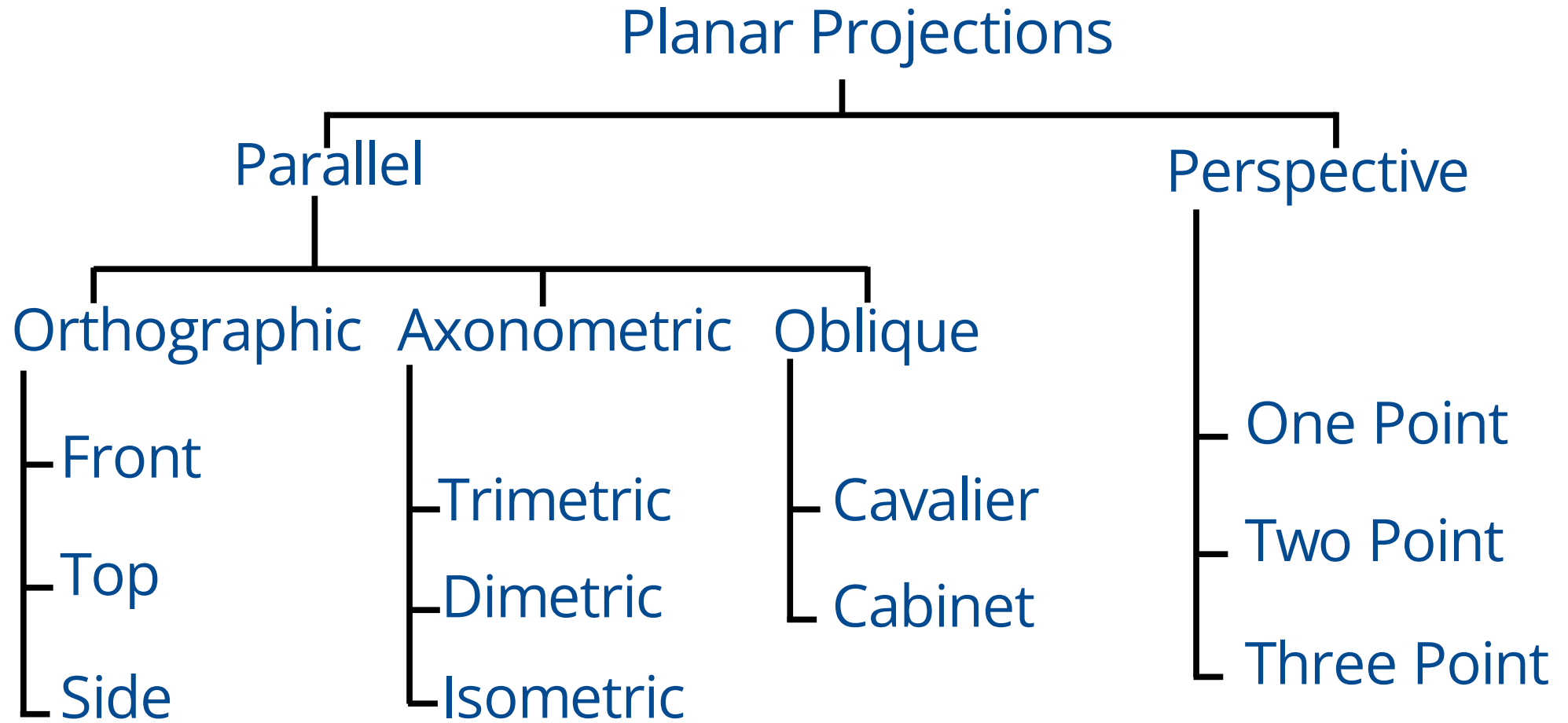


Department of Computer Science and Engineering
IIT Bombay

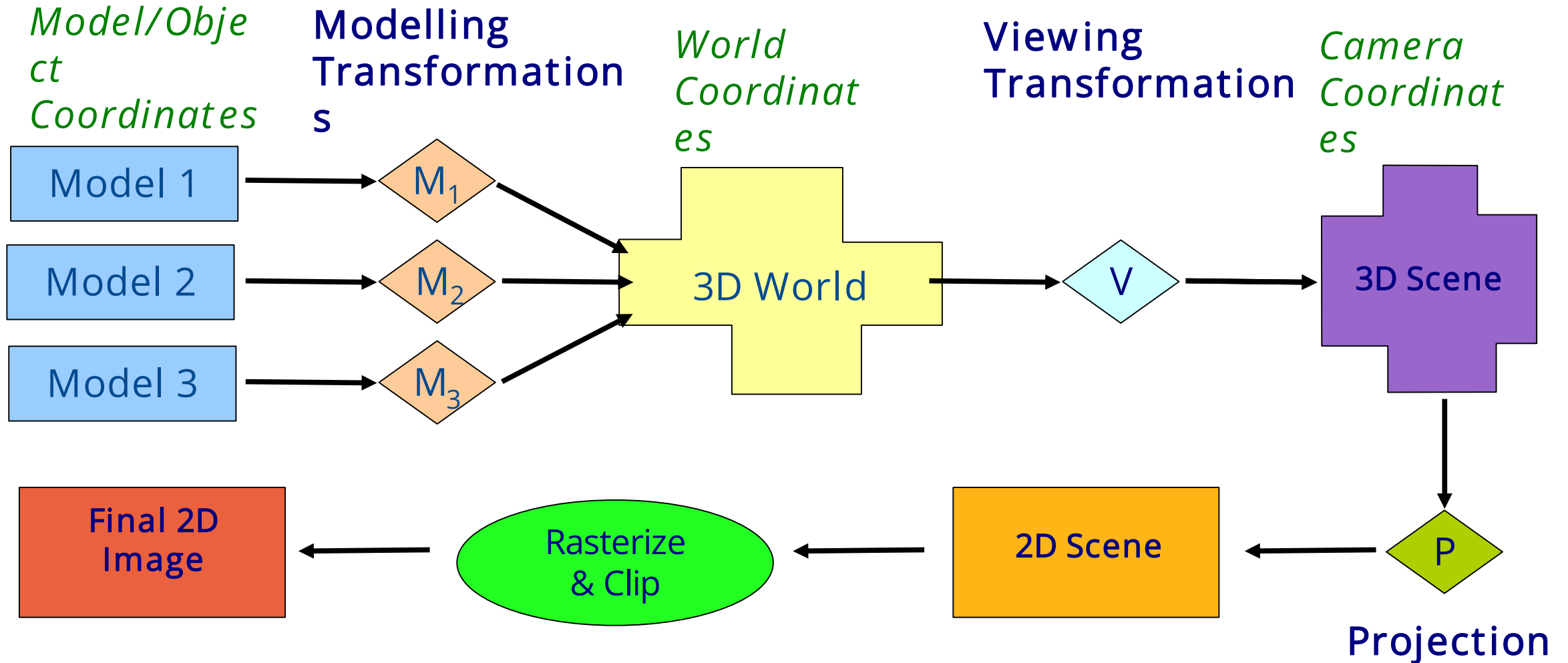


ViGIL
VISION, GRAPHICS,
IMAGING & LEARNING

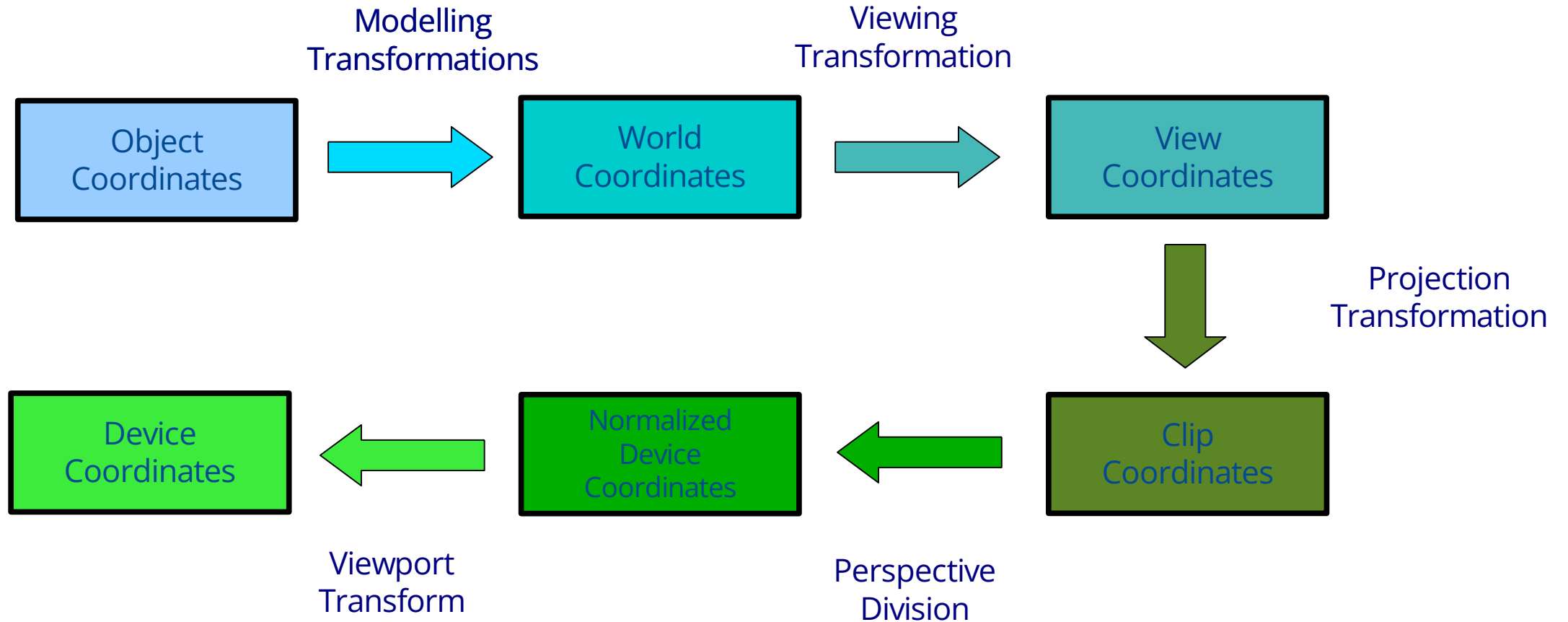
Taxonomy



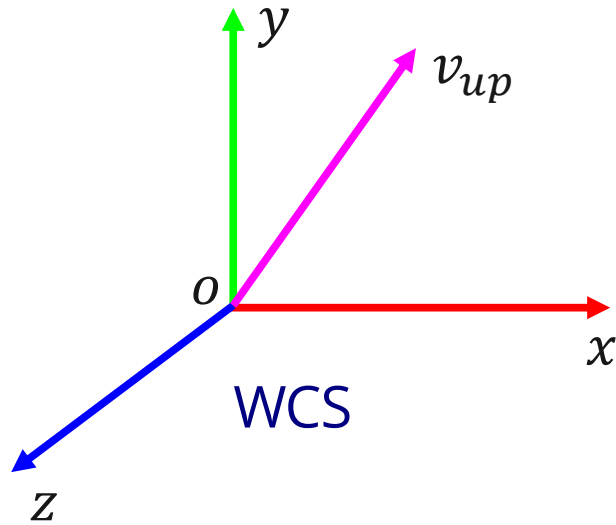
The Modeling-Viewing Pipeline



The Modeling-Viewing Pipeline



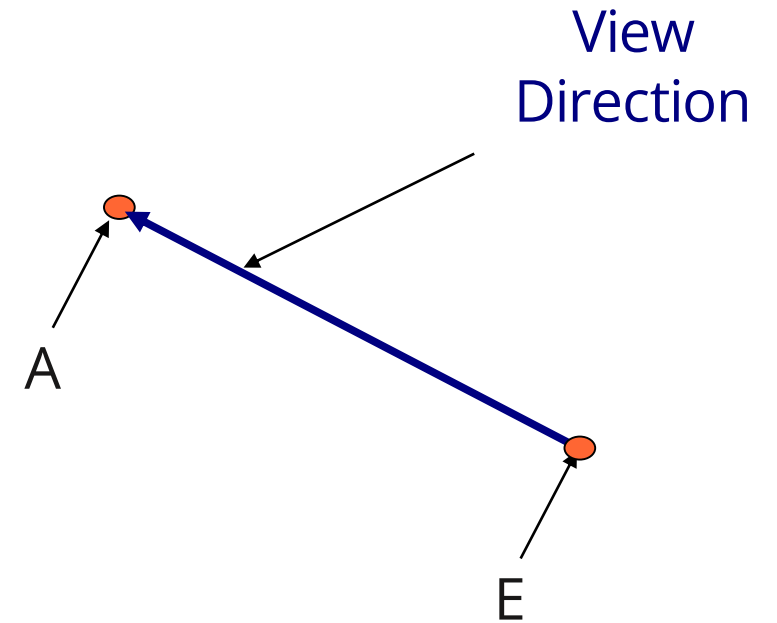
Viewing Transformation



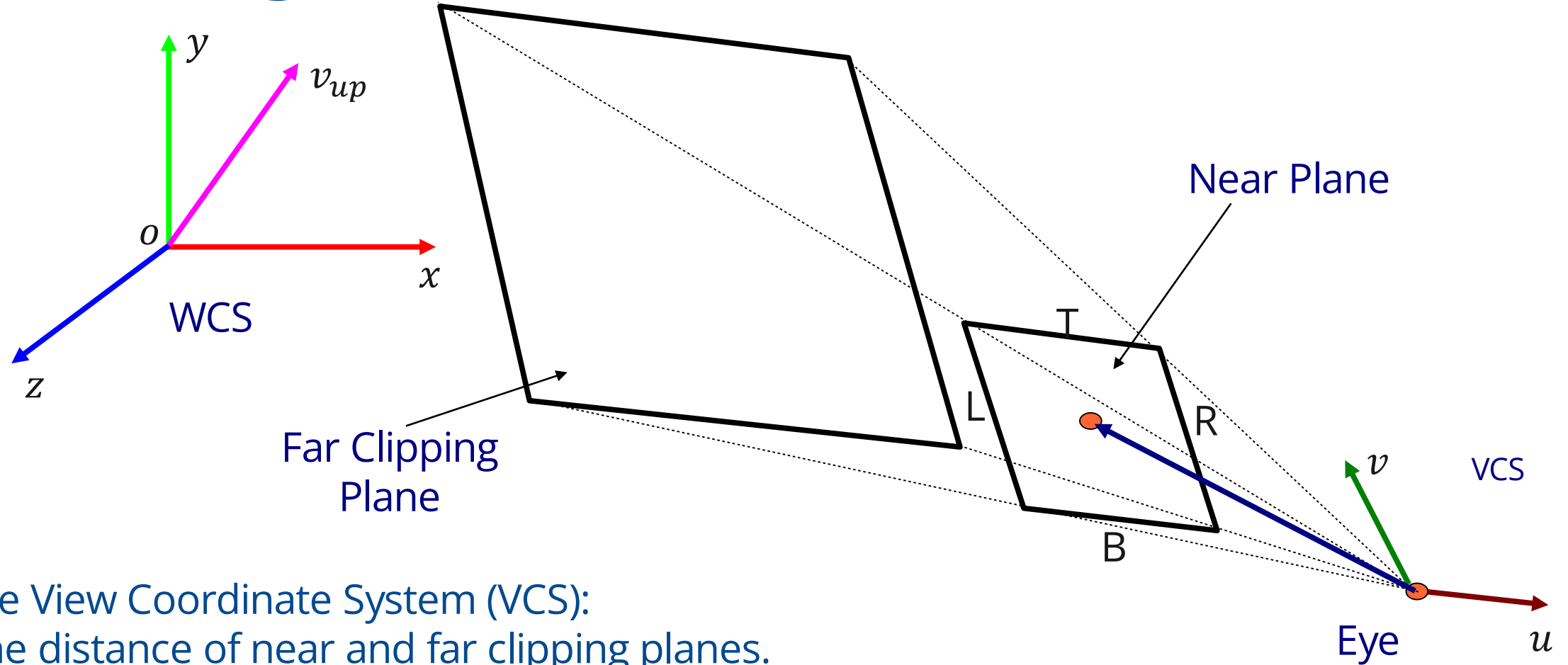
Given:

1. In the World Coordinate System (WCS):

- a) Position of the Eye (E)
- b) The lookat point (A)
- c) The up vector, v_{up}



Viewing Transformation



Given:

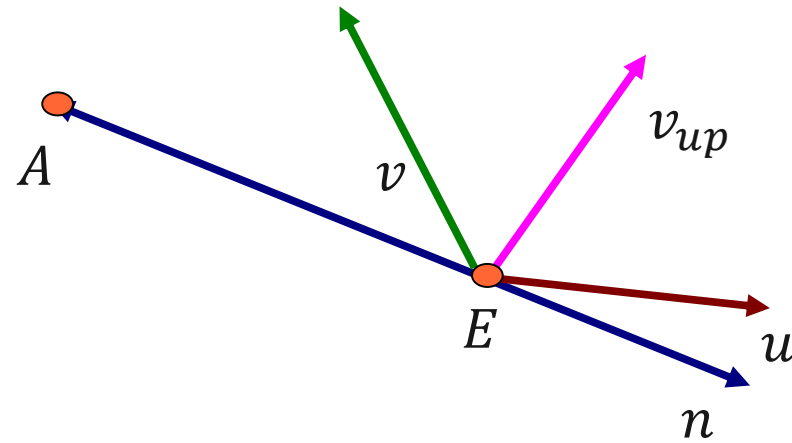
1. In the View Coordinate System (VCS):
 - a) The distance of near and far clipping planes.
 - b) Extents of the near plane L, R, T, B.

Defining the VCS

$$n = \frac{-(A - E)}{\|A - E\|}$$

$$u = \frac{v_{up} \times n}{\|v_{up} \times n\|}$$

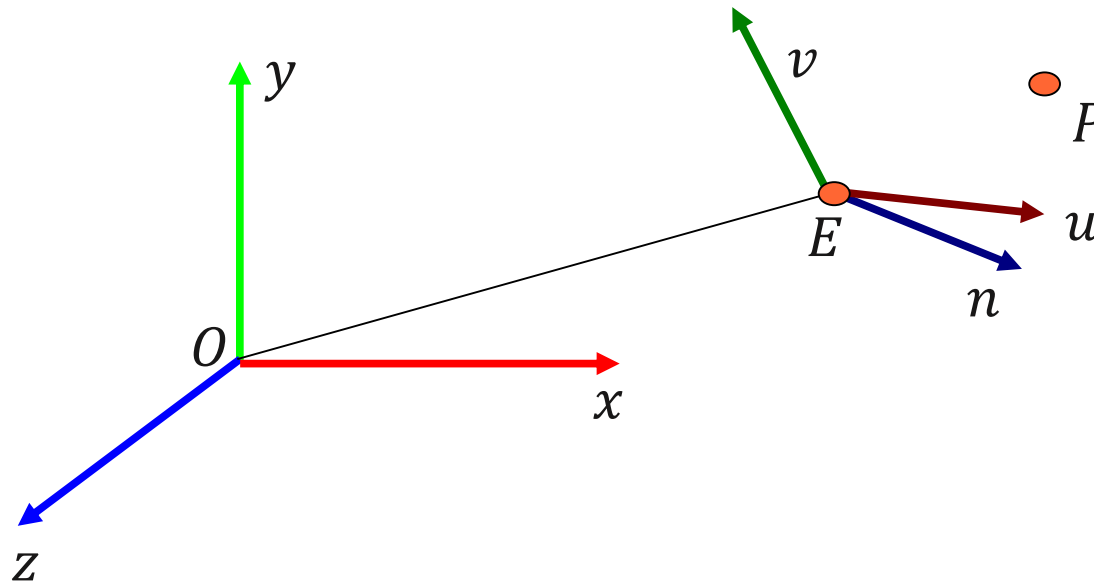
$$v = n \times u$$



From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

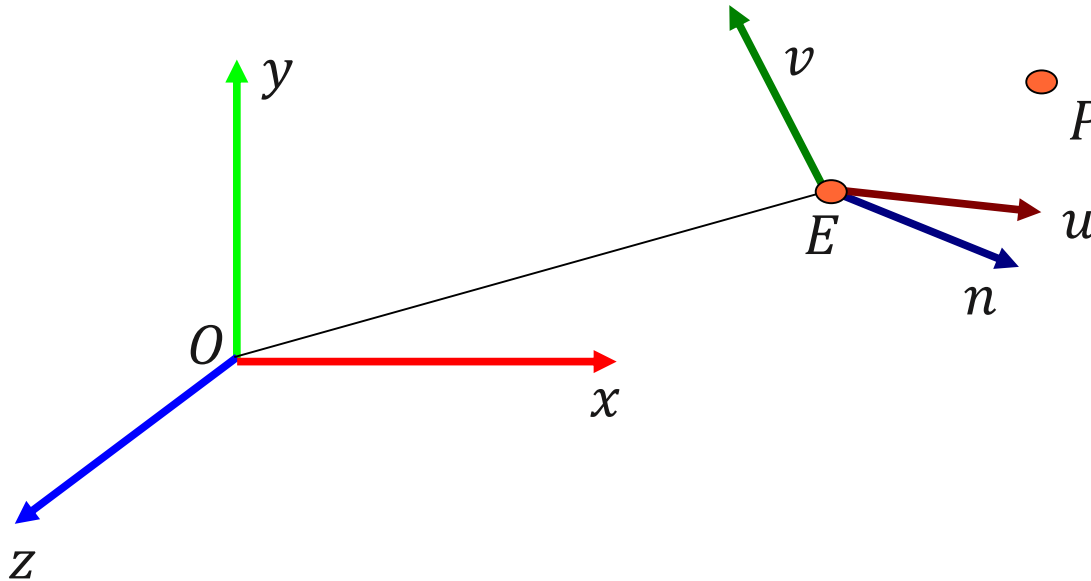


If the point P has coordinates
 (x, y, z) in WCS
 (a, b, c) in VCS
then -

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

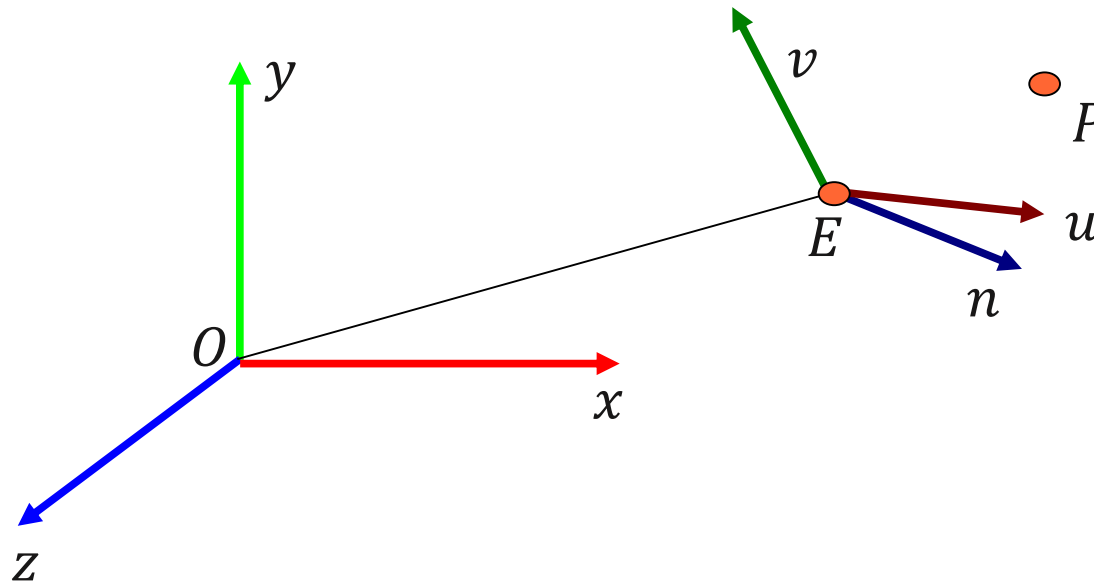


$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = [\vec{u} \vec{v} \vec{n}] \cdot \begin{bmatrix} a \\ b \\ c \end{bmatrix} + \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

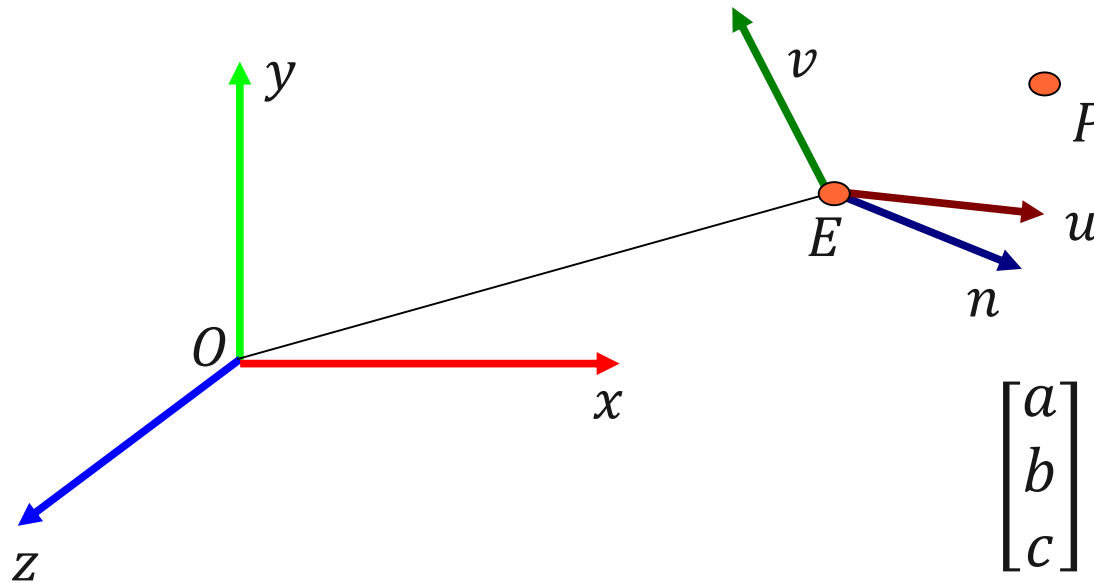


$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} u_x & v_x & n_x \\ u_y & v_y & n_y \\ u_z & v_z & n_z \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \end{bmatrix} + \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

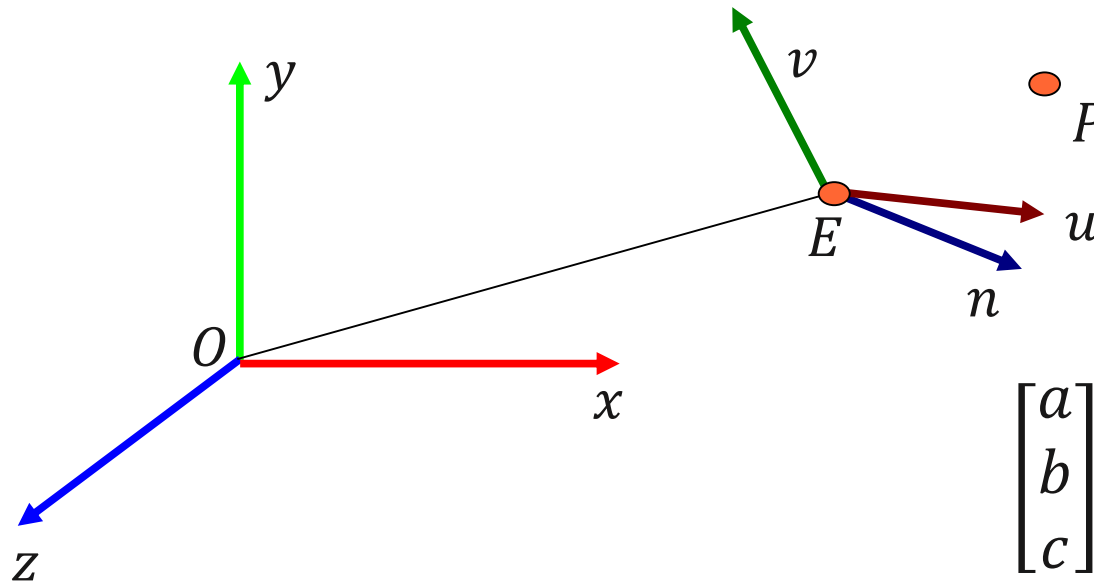


$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} u_x & v_x & n_x \\ u_y & v_y & n_y \\ u_z & v_z & n_z \end{bmatrix}^{-1} \cdot \left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} - \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix} \right)$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

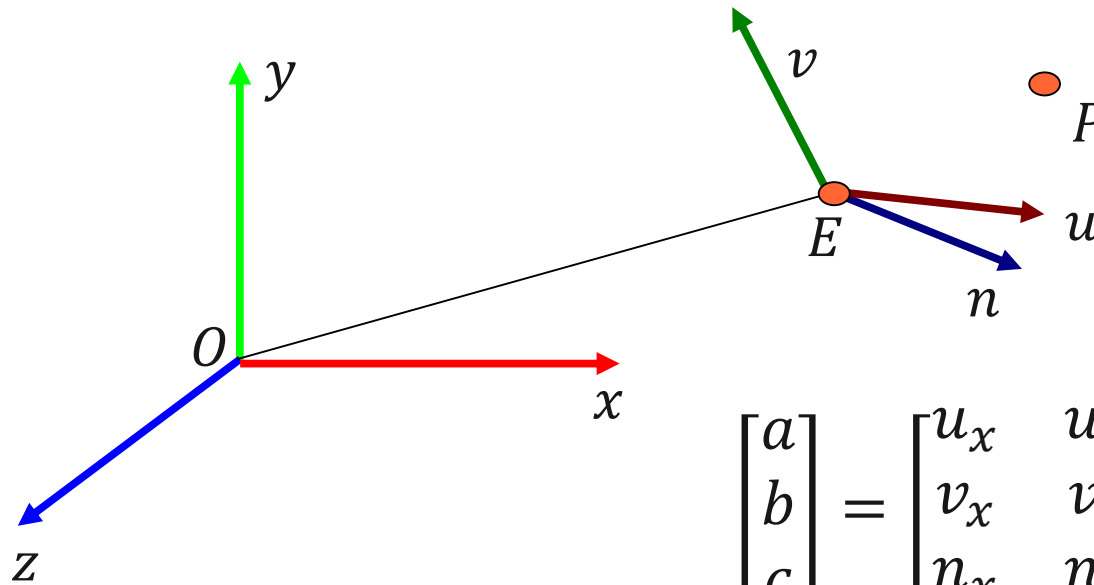


$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} u_x & v_x & n_x \\ u_y & v_y & n_y \\ u_z & v_z & n_z \end{bmatrix}^T \cdot \left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} - \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix} \right)$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

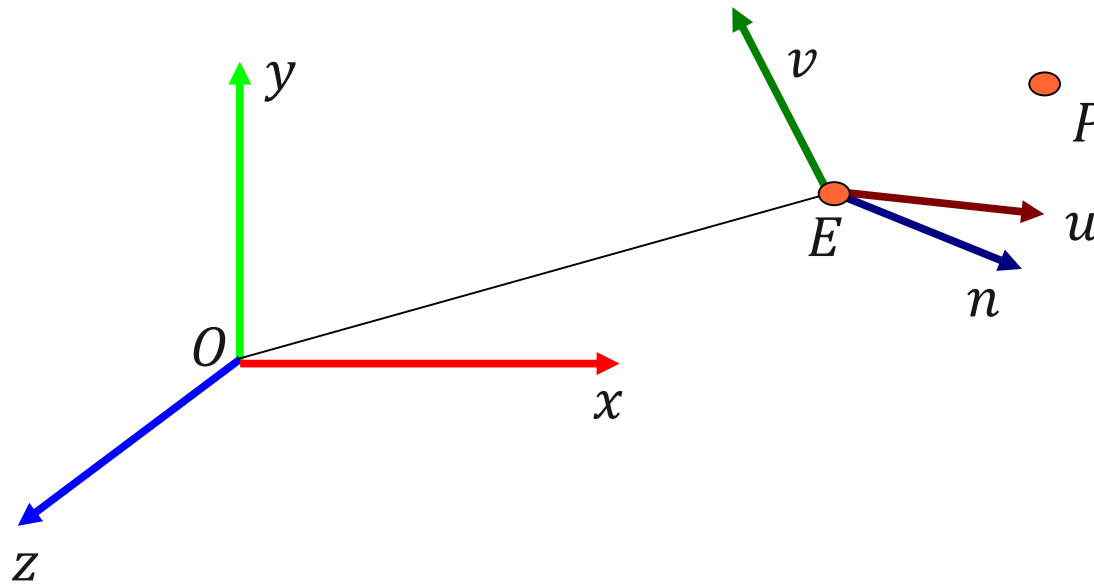


$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \\ n_x & n_y & n_z \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} - \begin{bmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \\ n_x & n_y & n_z \end{bmatrix} \cdot \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

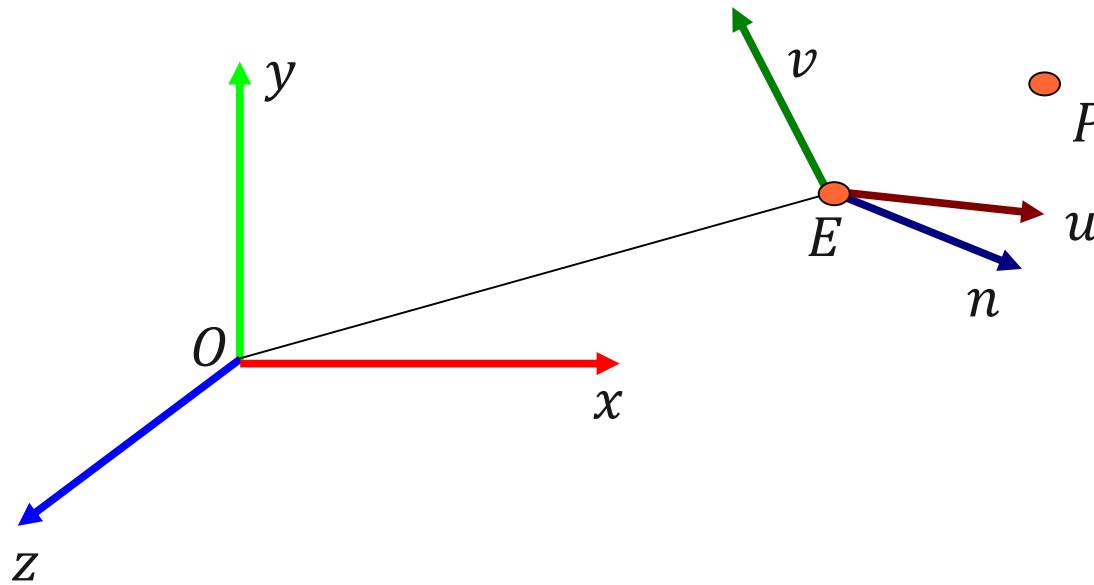


$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} \vec{u} \cdot \overrightarrow{OP} \\ \vec{v} \cdot \overrightarrow{OP} \\ \vec{n} \cdot \overrightarrow{OP} \end{bmatrix} - \begin{bmatrix} \vec{u} \cdot \overrightarrow{OE} \\ \vec{v} \cdot \overrightarrow{OE} \\ \vec{n} \cdot \overrightarrow{OE} \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

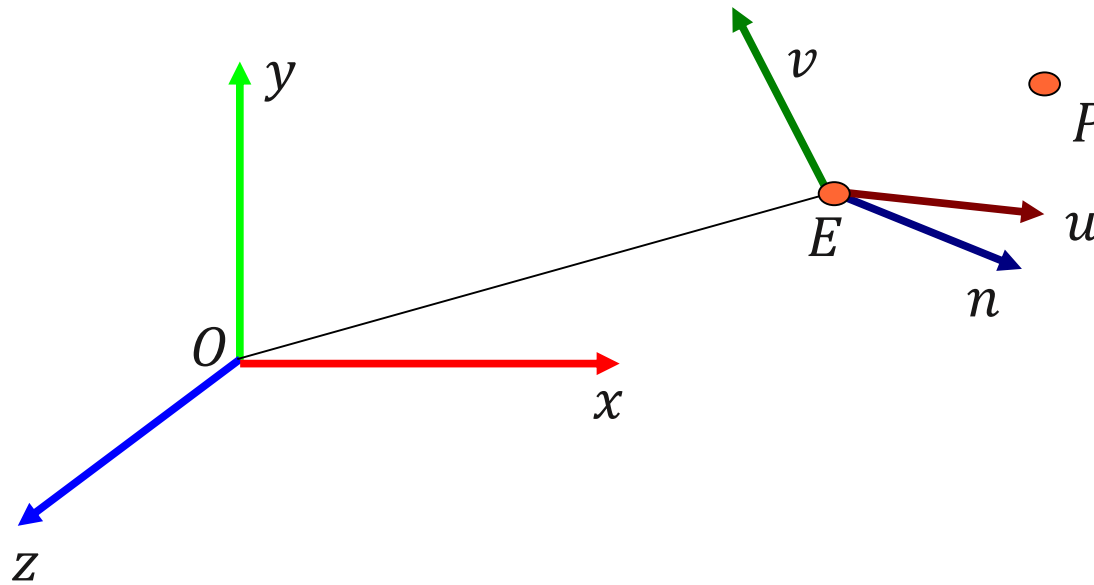


$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} \vec{u} \cdot \overrightarrow{OP} \\ \vec{v} \cdot \overrightarrow{OP} \\ \vec{n} \cdot \overrightarrow{OP} \end{bmatrix} + \begin{bmatrix} e_x' \\ e_y' \\ e_z' \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

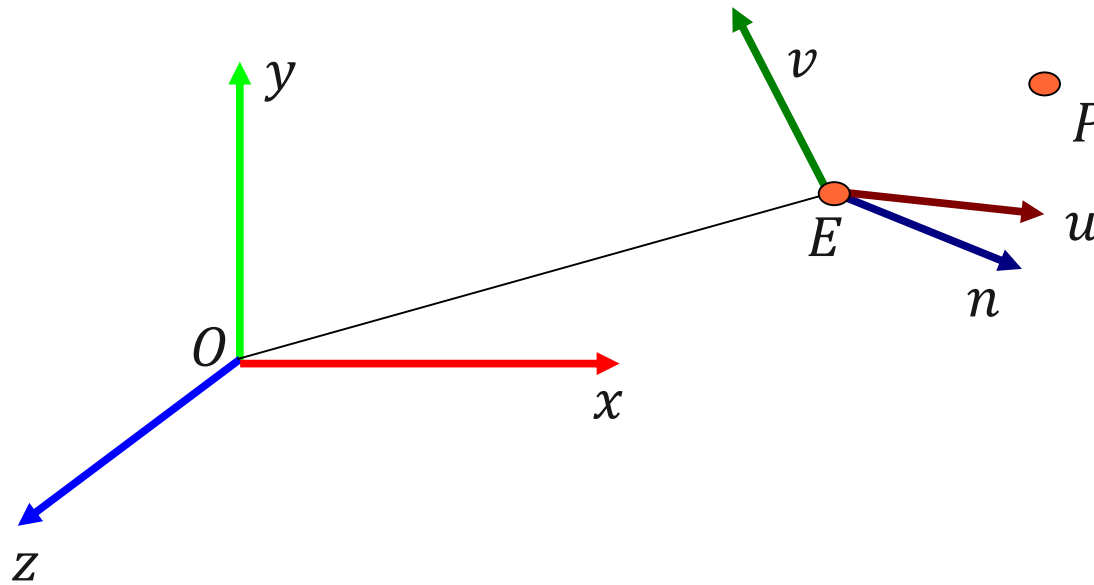


$$\begin{bmatrix} a \\ b \\ c \\ 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & e_x' \\ v_x & v_y & v_z & e_y' \\ n_x & n_y & n_z & e_z' \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

From WCS to VCS

The viewing transformation should:

- $x \rightarrow u$
- $y \rightarrow v$
- $z \rightarrow n$
- Map the origin of the WCS, O to the Eye, E

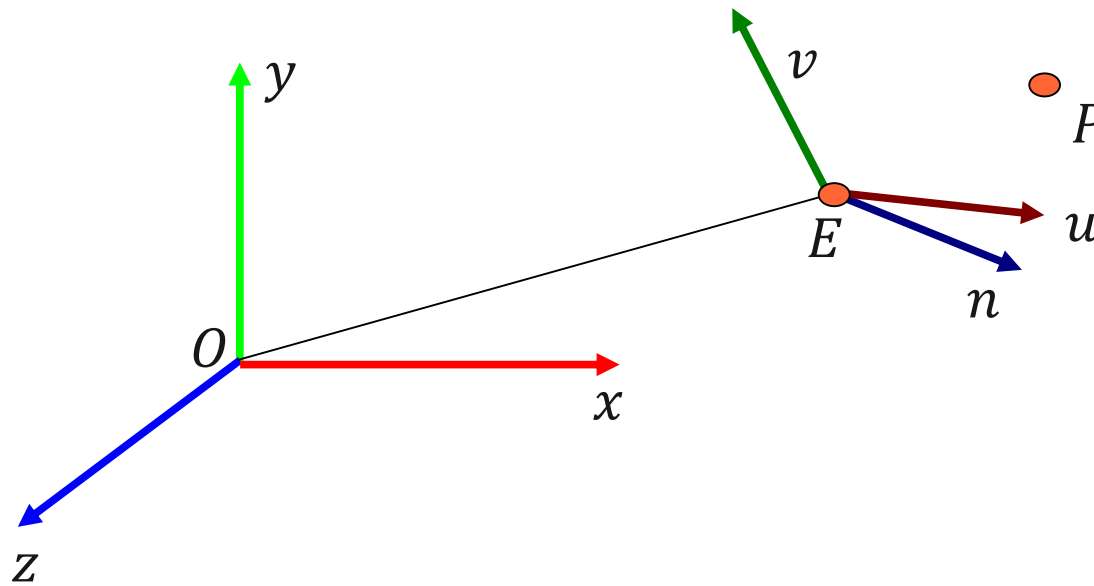


$$\begin{bmatrix} a \\ b \\ c \\ 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & e_x' \\ v_x & v_y & v_z & e_y' \\ n_x & n_y & n_z & e_z' \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The transformation matrix is labeled A_{wv} .

From WCS to VCS - OpenGL

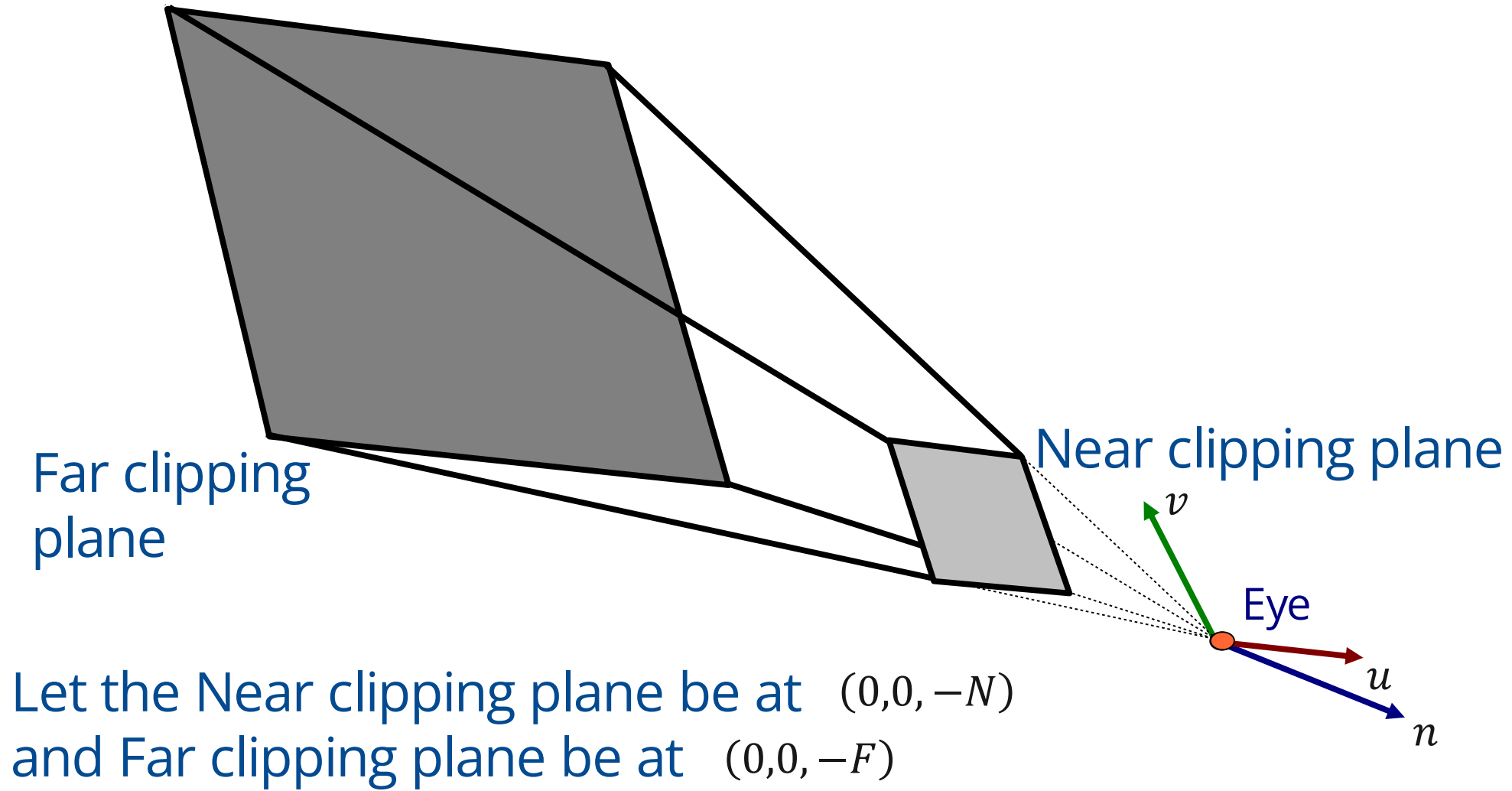
```
glm::lookAt(glm::vec3 eye, glm::vec3 lookat_pt, glm::vec3 upvec);
```



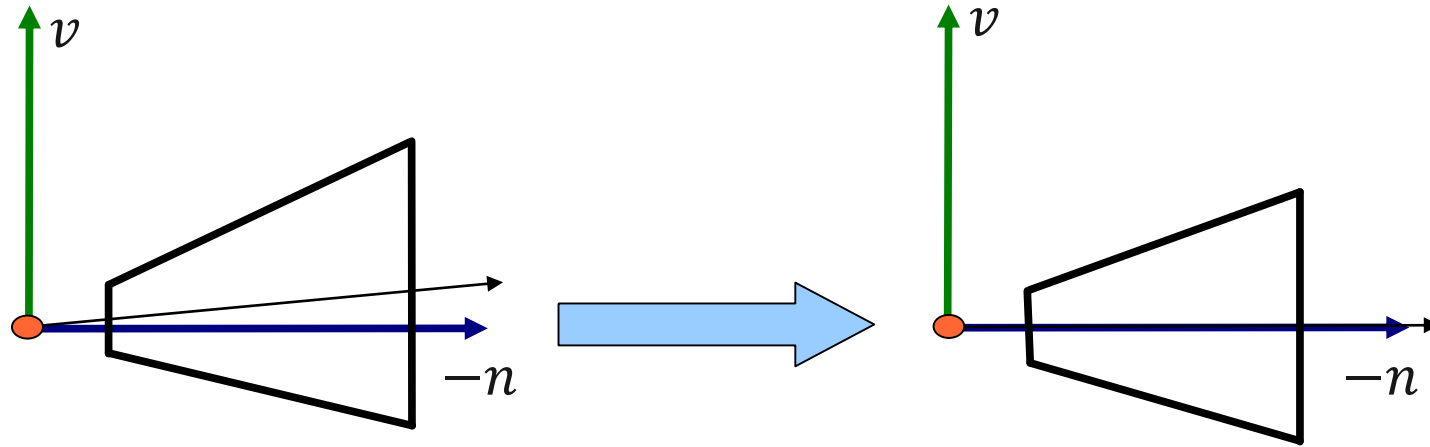
$$\begin{bmatrix} a \\ b \\ c \\ 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & e_x' \\ v_x & v_y & v_z & e_y' \\ n_x & n_y & n_z & e_z' \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The matrix is labeled A_{WV} with an arrow pointing to it. A red circle highlights the matrix, and a red arrow points from the label A_{WV} to the circle.

From WCS to VCS



From VCS to CCS

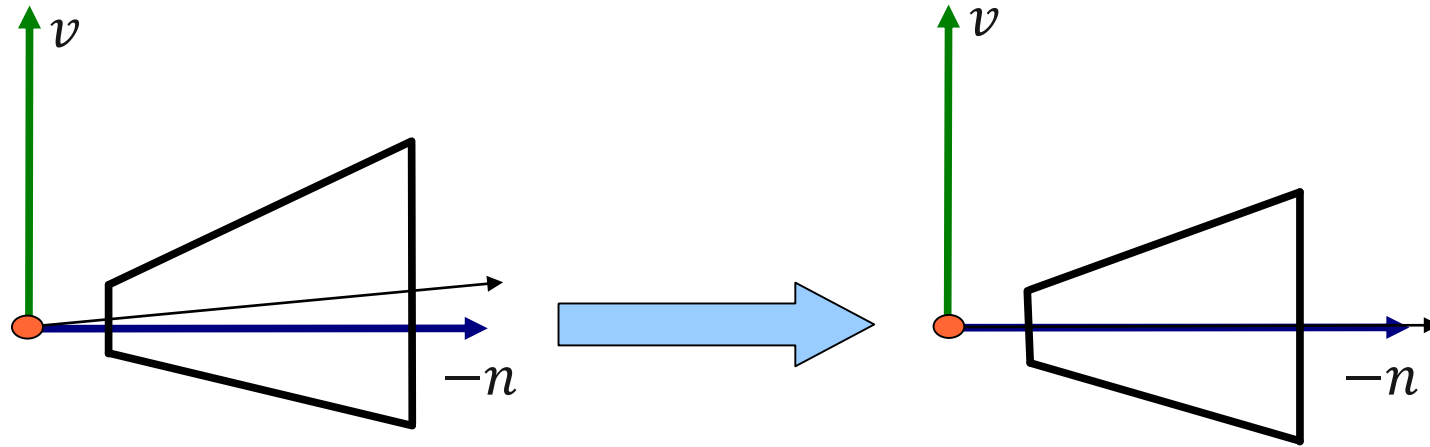


We shear the frustum so that the direction of projection aligns with the $-n$ axis and frustum becomes symmetrically aligned about it.

If the extents of the near plane are given by L, R, T, B then:

$$\begin{bmatrix} 0 \\ 0 \\ -N \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & Sh_{xz} & 0 \\ 0 & 1 & Sh_{yz} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} (R + L)/2 \\ (T + B)/2 \\ -N \\ 1 \end{bmatrix}$$

From VCS to CCS



We shear the frustum so that the direction of projection aligns with the $-n$ axis and frustum becomes symmetrically aligned about it.

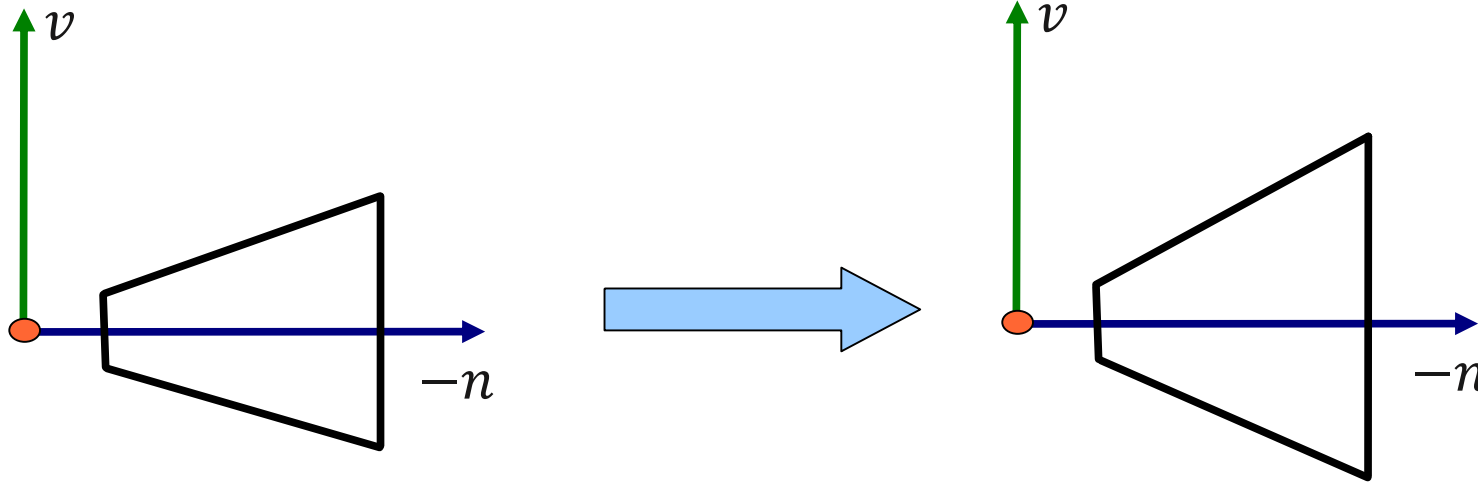
If the extents of the near plane are given by L, R, T, B then:

$$0 = (R + L)/2 - Sh_{xz} \cdot N \Rightarrow Sh_{xz} = \frac{R + L}{2N}$$

$$0 = (T + B)/2 - Sh_{yz} \cdot N \Rightarrow Sh_{yz} = \frac{T + B}{2N}$$

$$Sh = \begin{bmatrix} 1 & 0 & \frac{(R + L)}{2N} & 0 \\ 0 & 1 & \frac{(T + B)}{2N} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From VCS to CCS



Now we scale along u and v so that we get as the top/bottom and left/right side faces of the frustum.

$$u = \pm n, v = \pm n$$

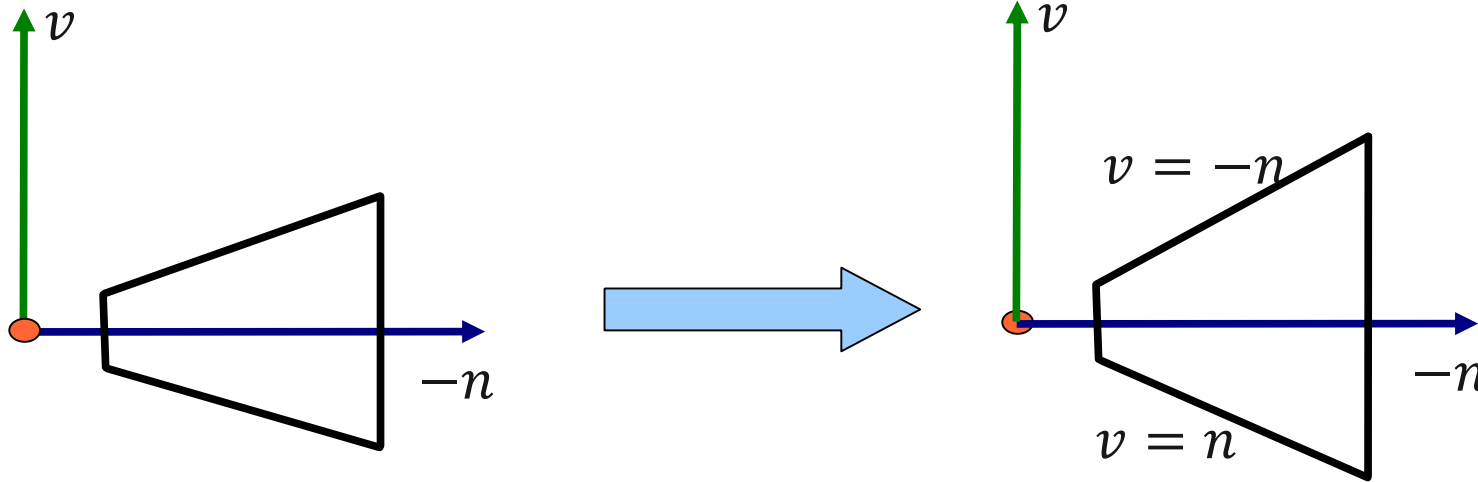
After shearing the point $(L, B, -N)$ becomes

$$\left(\frac{-(R - L)}{2}, \frac{-(T - B)}{2}, -N \right)$$

After shearing the point $(R, T, -N)$ becomes

$$\left(\frac{(R - L)}{2}, \frac{(T - B)}{2}, -N \right)$$

From VCS to CCS



Now we scale along u and v so that we get as the top/bottom and left/right side faces of the frustum.

$$u = \pm n, v = \pm n$$

So the scaling matrix should map

$$\left(\frac{\pm(R - L)}{2}, \frac{\pm(T - B)}{2}, -N \right) \text{ to } (\pm N, \pm N, -N)$$

$$Sc = \begin{bmatrix} \frac{2N}{(R - L)} & 0 & 0 & 0 \\ 0 & \frac{2N}{(T - B)} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From VCS to CCS

- As the final step we transform the frustum to a canonical frustum using a transformation is called a **projection normalization**.
- It is equivalent to doing a perspective transform.
- It brings us to the **Clip Coordinate System** or **CCS**.

The sides of the frustum have the following form after projection normalization

$$u = w, u = -w, v = w, v = -w, n = -w, n = w$$

$$(\pm N, \pm N, -N, 1) \text{ maps to } (\pm N, \pm N, -N, N)$$

$$(\pm F, \pm F, -F, 1) \text{ maps to } (\pm F, \pm F, F, F)$$

$$Nm = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-(F+N)}{(F-N)} & \frac{-2FN}{(F-N)} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

From VCS to CCS

- As the final step we transform the frustum to a canonical frustum using a transformation is called a **projection normalization**.
- It is equivalent to doing a perspective transform.
- It brings us to the **Clip Coordinate System** or **CCS**.

The sides of the frustum have the following form after projection normalization

$$u = w, u = -w, v = w, v = -w, n = -w, n = w$$

$(\pm N, \pm N, -N, 1)$ maps to $(\pm N, \pm N, -N, N)$

$(\pm F, \pm F, -F, 1)$ maps to $(\pm F, \pm F, F, F)$

$$Nm = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-(F+N)}{(F-N)} & \frac{-2FN}{(F-N)} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Note that the near and far planes are now on opposite sides of the origin. This is because the origin shifts due to the translation along the n-axis.

From VCS to CCS

View-Frustum Clipping is done in the CCS – homogenous co-ordinates.

$$u = w, u = -w, v = w, v = -w, n = -w, n = w$$

Cohen-Sutherland Clipping can be applied with the following criteria that a point (u, v, n) is visible if (can you design the bitcodes for this ?)

$$-w \leq u \leq w$$

$$-w \leq v \leq w$$

$$-w \leq n \leq w$$

Why is the clipping done in homogenous coordinates and not Euclidean coordinates?

From VCS to CCS

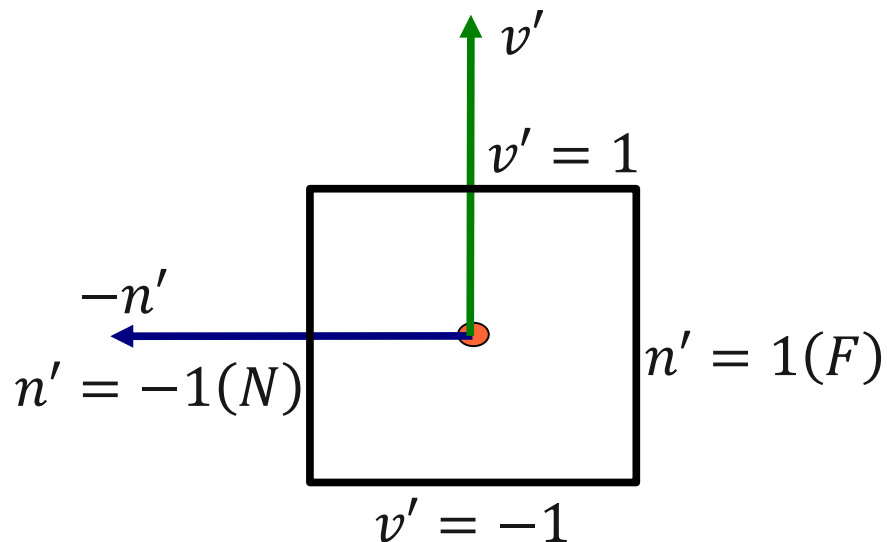
So the complete transformation is:

$$A_{vc} = Nm.Sc.Sh$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{-(F+N)}{(F-N)} & \frac{-2FN}{(F-N)} \\ 0 & 0 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \frac{2N}{(R-L)} & 0 & 0 & 0 \\ 0 & \frac{2N}{(T-B)} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & \frac{(R+L)}{2N} & 0 \\ 0 & 1 & \frac{(T+B)}{2N} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From CCS to NDCS

$$A_{vc} = \begin{bmatrix} \frac{2N}{(R-L)} & 0 & \frac{(R+L)}{(R-L)} & 0 \\ 0 & \frac{2N}{(T-B)} & \frac{(T+B)}{(T-B)} & 0 \\ 0 & 0 & \frac{-(F+N)}{(F-N)} & \frac{-2FN}{(F-N)} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$



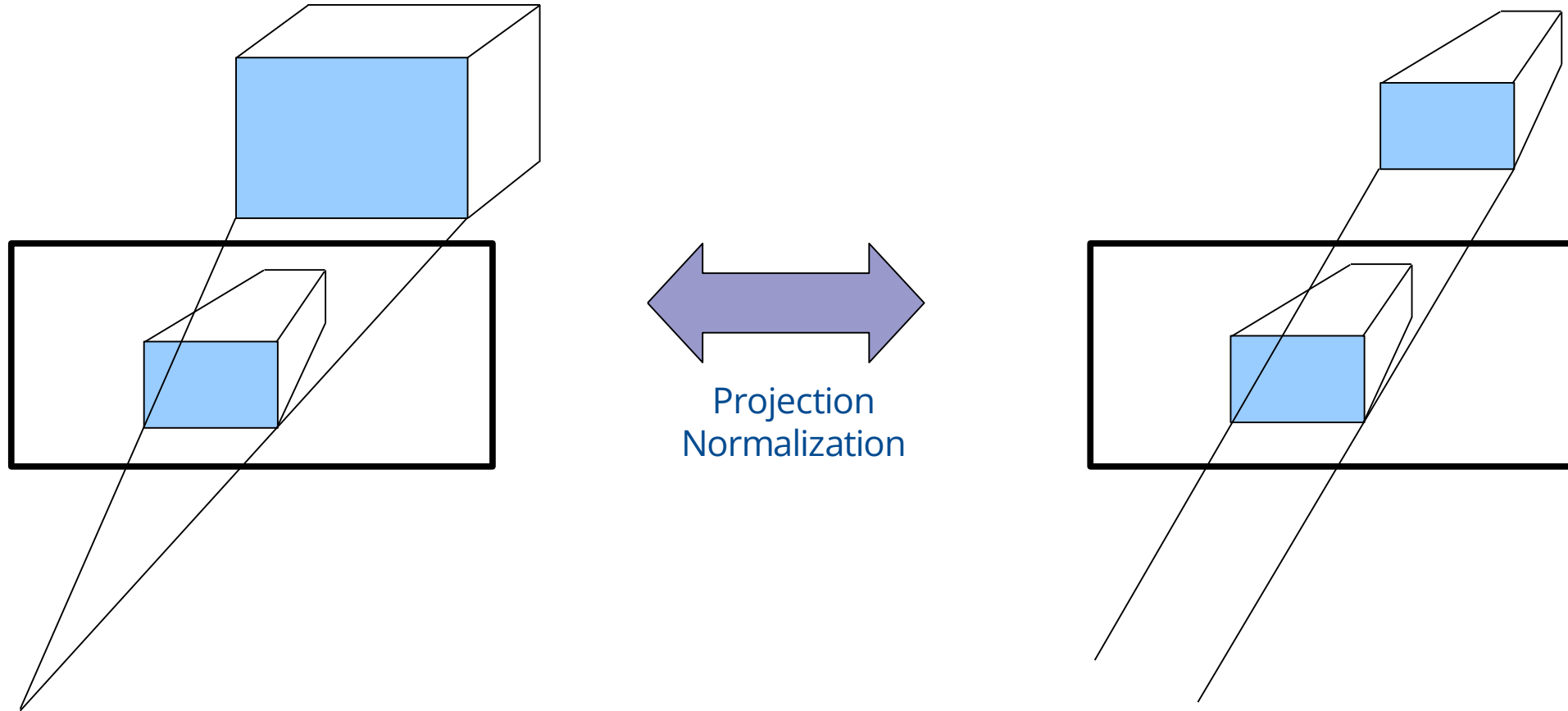
CCS coordinates retain the homogenous coordinate.

This is followed by a ***perspective divide*** stage where the coordinates are divided by the 'w' coordinate.

That puts all the coordinates within the *normalized +/- 1 cube*.

This coordinates system is called the **Normalized Device Coordinate System** or **NDCS**.

Projection Normalization



An Orthographic projection of a distorted object can be the same as a Perspective projection of the undistorted object.

From VCS to NDCS - OpenGL

`glm::frustum(L, R, B, T, N, F)`

or

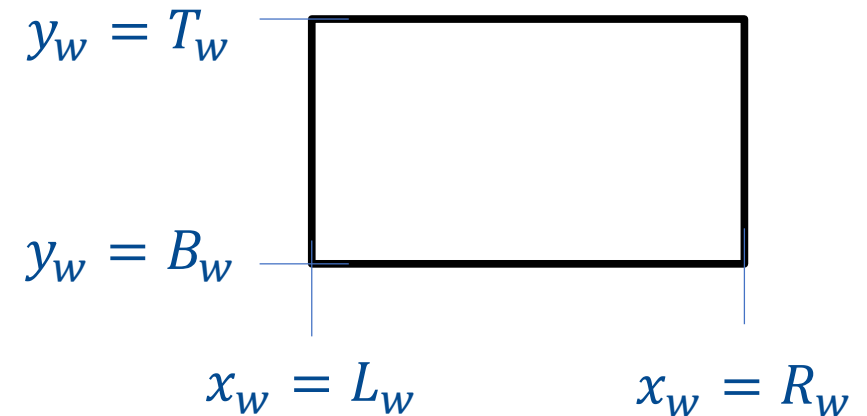
`glm::perspective(fovy, aspect, N, F)`

N and F give the distance of Near and Far clipping planes from the Eye and must be positive numbers and must not be equal.

From NDCS to DCS

Now we need to map the NDCs to Device or Window coordinates. This is done by the viewport transformation. If the aspect of the viewport is not the same as that of the view frustum then the image is distorted.

$$x_w = \frac{(x + 1) \cdot (R_w - L_w)}{2} + L_w$$
$$y_w = \frac{(y + 1) \cdot (T_w - B_w)}{2} + B_w$$
$$z_w = \frac{(z + 1)}{2}$$



From NDCS to DCS- OpenGL

`glDepthRange( $N_f$ ,  $F_f$ )` : Default range is 0 to 1

`glViewport(x, y, width, height)`

From VCS to NDCS - Orthographic

An Orthographic frustum is fully specified by L, R, T, B, N and F as well. Here there is no need to shear the frustum to make it symmetric – a translation can center it on the n-axis. This is followed by a scaling to transform the frustum to the canonical frustum. No projection normalization is required.

$$A_{vc} = Sc.T$$
$$= \begin{bmatrix} \frac{2}{(R-L)} & 0 & 0 & \frac{(R+L)}{(R-L)} \\ 0 & \frac{2}{(T-B)} & 0 & \frac{(T+B)}{(T-B)} \\ 0 & 0 & \frac{-2}{(F-N)} & \frac{(F+N)}{(F-N)} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

glm::ortho(L, R, B, T, N, F)

or

glm::ortho(L, R, B, T)

Why carry the depth through?

Intuition: Visibility computation or Hidden Surface Removal

Algorithm: The Z-Buffer Algorithm

OpenGL:

```
glClear(GL_DEPTH_BUFFER_BIT);  
glEnable(GL_DEPTH_TEST);
```